Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 10

Team No.: 21

Project Title: Linux Memory-Mapped File Management System

Team Members

Roll Number	Student Name	Signature
2520090149	Sai Snehith	
2520090114	Harshith Reddy	
2520090122	Rithesh	

1. Abstract

*The **Linux Memory-Mapped File Management System** is a systems programming project designed to demonstrate efficient file management using memory mapping in a Linux environment. Traditional file handling methods use system calls such as read() and write() to transfer data between files and process memory. This project explores an alternative approach using the mmap() system call, which maps a file directly into the process's virtual address space.*

*The proposed system will provide functionalities for creating, opening, mapping, reading, modifying, synchronizing, resizing, and unmapping files. It will demonstrate important Operating Systems concepts such as **virtual memory, process address space, memory mapping, file I/O, page management, system calls, and file synchronization**. Linux/POSIX APIs such as open(), mmap(), munmap(), msync(), ftruncate(), and close() will be used for implementation.*

The system will allow users to observe how modifications made to memory-mapped regions can be synchronized with the corresponding file. It will also demonstrate the advantages of memory-mapped file access, particularly for efficient handling of large files. The expected outcome is a functional Linux-based application that provides a practical understanding of memory-mapped files and their interaction with the operating system's memory and file-management mechanisms.

2. Problem Statement

Traditional file access methods require repeated system calls such as read() and write() to transfer data between secondary storage and process memory. This can introduce additional overhead, especially when working with large files or frequently accessed data. There is a need to understand and demonstrate a more efficient file-access mechanism using memory mapping. The project aims to develop a Linux-based Memory-Mapped File Management System that allows users to create, open, map, access, modify, synchronize, and unmap files while demonstrating how the operating system manages virtual memory and file-backed memory regions.

---

3. Objectives

1. To implement file management using the Linux mmap() system call.

2. To demonstrate the interaction between virtual memory and file storage through memory-mapped files.

3. To provide operations such as file creation, opening, reading, writing, synchronization, resizing, and unmapping

4. To study and demonstrate relevant Linux system calls, POSIX APIs, memory management, and file I/O mechanisms.

4. Proposed Methodology

The project will be implemented in a Linux environment using C/C++. The system will first create or open a selected file using standard Linux file-handling system calls. The file will then be mapped into the process's virtual address space using the mmap() system call. Users will be able to read and modify the mapped data directly through memory operations. Changes made to the mapped region will be synchronized with the file using msync(). The system will also support file resizing using ftruncate() and will release the mapped memory using munmap() after the required operations are completed. Appropriate error handling will be included for invalid files, mapping failures, permission issues, and other system-call errors. The overall flow will demonstrate the relationship between process memory, virtual memory mapping, and persistent file storage in Linux.

5. Operating Systems Concepts / Linux APIs Used

(List the relevant Operating Systems concepts, Linux system calls, POSIX APIs, or mechanisms that will be used.)

OS Concept / Linux API / System Call	Purpose in the Project
Virtual Memory	Understand how files are mapped into a process's virtual address space.
open()	Opens or creates the file for memory mapping.
mmap()	Maps the file into the process's memory address space.
munmap()	Removes the mapped file region from the process's address space.
msync()	Synchronizes changes made in memory with the file on storage.
ftruncate()	Changes the size of the file when required.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520090149	Sai Snehith	Design and implementation of memory-mapped file operations
2520090114	Harshith Reddy	Linux system calls and file-management module
2520090122	Rithesh	Memory management and synchronization module

7. Tools / Platforms / Software Used

(List the programming languages, operating system, libraries, tools, and software planned for the project.)

Tool / Platform / Software	Purpose
Linux / Ubuntu	Operating environment for implementing and testing the system
C / C++	Programming language for system-level implementation
GCC / G++	Compilation of the project source code
Linux System Calls / POSIX APIs	File handling, memory mapping and synchronization
Git / GitHub	Version control and project repository

8. Expected Outcome

The project is expected to produce a functional Linux-based Memory-Mapped File Management System that allows users to create, open, map, read, modify, synchronize, resize, and unmap files. It will demonstrate the practical working of virtual memory, memory mapping, file I/O, and Linux system calls. The system will provide a clear understanding of how file data can be accessed through memory and how modifications are synchronized with persistent storage.

These choices align with the form's requirement to list relevant ***OS concepts/Linux APIs and their purposes****, followed by tools and the expected functionality/output.*

9. GitHub Repository

GitHub Repository Link: https://github.com/ritheshsavulla/OSSP/tree/KLH_CSIT_2026-27_BATCH-21_Linux-Memory

Repository Created: ____________________

☐ Yes ☐ No

Faculty Name and Signature

Faculty Name: Ragupathi

Signature: ____

Remarks: ____

Faculty Use

Project Approved: ___________________________________

☐ Yes ☐ No

Date: ____

Faculty Signature: ____